

EVALUASI TENGAH SEMESTER
MATA KULIAH PEMROGRAMAN KONTROLLER

**Implementasi Embedded Rust pada ESP32-S3 untuk Akuisisi Data
Sensor DHT22 dan MQ-2 dengan Circular Buffer, Moving Average,
Validasi Data, Decision Logic, dan Kontrol LED**

Dosen Pengampu: Ahmad Radhy, S.Si., M.Si.



Disusun oleh (Kelompok 8):

Khusnul Khuluq Ahmada (NRP: 2042241001)
Wildan Maulana Zulkha (NRP: 2042241059)

Kelas: 4C

PRODI D4 TEKNOLOGI REKAYASA INSTRUMENTASI
DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

1 Studi Literatur

Tabel 1 menyajikan ringkasan dari 15 literatur acuan yang dihimpun melalui pangkalan data Scopus dan Web of Science (rentang publikasi 2021-2026). Kajian komprehensif ini digunakan sebagai landasan teoretis untuk pengembangan rancangan arsitektur sistem.

Table 1: Ringkasan Literatur Acuan

No	Judul Artikel & Penulis (Tahun)	Metode	Hasil Utama	Future Work
1	Overview of Embedded Rust Operating Systems and Frameworks <i>Vandervelden et al., 2024</i>	Melakukan studi komparatif dan tinjauan literatur mengenai arsitektur sistem operasi tertanam (RTOS) yang dibangun menggunakan C dibandingkan dengan Rust.	Rust secara inheren mampu menjamin keamanan memori sejak fase kompilasi, mengungguli C yang rentan terhadap kebocoran memori. Namun, ekosistem Rust untuk periferal spesifik masih berada pada tahap adopsi awal.	Perlu adanya optimasi fitur asinkron (<i>async/await</i>) pada <i>framework</i> Rust untuk sistem tertanam guna meningkatkan efisiensi komputasi CPU.
2	Rust for Embedded Systems: Current State and Open Problems <i>Sharma, n.d., 2024</i>	Melakukan analisis statis terhadap 6.408 repositori perangkat lunak <i>embedded</i> Rust serta menyebarkan kuesioner kepada 225 pengembang.	Meskipun Rust menawarkan keamanan tinggi, pengembang masih sering bergantung pada blok kode <i>unsafe</i> saat mengakses register tingkat keras akibat keterbatasan kapabilitas <i>Hardware Abstraction Layer</i> (HAL).	Dibutuhkan pengembangan lapisan abstraksi register (driver kustom) yang lebih aman guna meminimalisasi penggunaan blok <i>unsafe</i> pada interaksi periferal.

Berlanjut ke halaman berikutnya...

Table 1 – Lanjutan dari halaman sebelumnya

No	Judul Artikel & Penulis (Tahun)	Metode	Hasil Utama	Future Work
3	FMP3+TECS/Rust: Memory-Safe Component Framework for Multiprocessor Embedded Systems <i>Yoshimura et al., 2026</i>	Mengembangkan arsitektur kerangka kerja komponen (TECS/Rust) yang terintegrasi dengan sistem operasi multiprosesor (FMP3 RTOS) guna mengamankan alokasi memori.	Arsitektur Rust terbukti efektif mencegah <i>data race</i> antar-inti (<i>core</i>) pada lingkungan multiprosesor dengan <i>overhead</i> eksekusi yang komparabel dengan C.	Implementasi sinkronisasi <i>task</i> secara asinkron pada unit <i>Dual-Core</i> (misal: ESP32-S3) menggunakan Rust direkomendasikan untuk menghindari tabrakan memori antara proses akuisisi dan komunikasi.
4	Continuous energy consumption measure approach using a DMA double-buffering technique <i>Hdez et al., 2021</i>	Mengaplikasikan modul <i>Direct Memory Access</i> (DMA) dengan teknik <i>double-buffering</i> dan <i>pipeline processing</i> untuk mereduksi intervensi CPU saat akuisisi ADC kontinu.	Pendekatan ini berhasil mengelem-inasi <i>bottleneck</i> aliran data pada frekuensi sampel tinggi, sehingga CPU dapat difokuskan pada operasi kalkulasi matematis.	Integrasi ADC frekuensi tinggi dengan sistem DMA <i>double-buffering</i> pada ESP32-S3 perlu dilakukan agar beban komputasi CPU tidak tersita oleh proses akuisisi sensor.
5	A Low-Latency Optimization of a Rust-Based Secure Operating System for Embedded Devices <i>Culic et al., 2022</i>	Menerapkan teknologi <i>extended Berkeley Packet Filter</i> (eBPF) pada sistem operasi tertanam (Tock OS) untuk resolusi <i>hardware interrupts</i> secara langsung.	Optimasi ini berhasil menekan <i>interrupt latency</i> hingga 60 μ s dengan tetap mempertahankan integritas jaminan keamanan memori yang ditawarkan oleh Rust.	Substitusi <i>Interrupt Service Routines</i> (ISR) konvensional dengan arsitektur <i>event-driven</i> berbasis <i>async executor</i> dianjurkan untuk memangkas latensi periferai.

Berlanjut ke halaman berikutnya...

Table 1 – Lanjutan dari halaman sebelumnya

No	Judul Artikel & Penulis (Tahun)	Metode	Hasil Utama	Future Work
6	An IoT General-Purpose Sensor Board for Enabling Remote Aquatic Environmental Monitoring <i>Trevathan et al., 2021</i>	Mendesain modul sensor IoT <i>general-purpose</i> yang mengonsolidasikan beragam instrumen pemantau kualitas air melalui antar-muka serial standar (I2C dan SPI).	Prototipe mampu mengakuisisi data lingkungan secara <i>real-time</i> dengan efisiensi daya tinggi, kendati masih rentan mengalami <i>lockup</i> apabila koneksi sensor terputus mendadak.	Pengembangan mekanisme <i>auto-recovery</i> pada <i>state machine</i> bus I2C/SPI dengan Rust sangat esensial untuk memulihkan sistem jika komunikasi perangkat keras mengalami distorsi.
7	Energy-Age of Incorrect Information Trade-off with Timer-Based Sleep-Wake Scheduling <i>Gursoy et al., 2024</i>	Memformulasikan skema penjadwalan <i>sleep-wake</i> presisi berbasis pewaktu untuk menyeimbangkan konsumsi daya transmisi dan tingkat kebaruan informasi data sensor.	Distribusi interval waktu yang dioptimasi secara analitis terbukti memperpanjang umur baterai perangkat tanpa mengorbankan relevansi informasi (data tidak kedaluwarsa).	Penerapan siklus pewaktu <i>sleep/wake-up</i> yang adaptif pada ESP32-S3, di mana jedanya menyesuaikan diri secara otomatis terhadap laju dinamika nilai sensor.
8	Watchdog Timer for Fault Tolerance in Embedded Systems <i>Automatisés et al., 2024</i>	Menggabungkan arsitektur <i>Watchdog Timer</i> (WDT) dengan algoritma <i>Earliest Deadline First</i> (EDF) guna mendeteksi anomali pada sistem operasi waktu nyata.	Skema WDT ini secara drastis meningkatkan ketahanan sistem melalui kemampuannya merestorasi proses yang malfungsi tanpa merugikan eksekusi rutin berprioritas tinggi.	Diperlukan implementasi <i>software watchdog timer</i> granular yang mampu mengawasi siklus <i>asynchronous task</i> pada setiap <i>core</i> secara independen.

Berlanjut ke halaman berikutnya...

Table 1 – Lanjutan dari halaman sebelumnya

No	Judul Artikel & Penulis (Tahun)	Metode	Hasil Utama	Future Work
9	CHERIoT: Complete Memory Safety for Embedded Devices <i>Amar et al., n.d., 2024</i>	Mengadaptasikan paradigma kapabilitas tingkat keras (CHERI) bersama RTOS khusus untuk menegakkan isolasi (<i>compartmentalization</i>) aplikasi secara ketat.	Pendekatan ini sukses membentuk partisi keamanan memori absolut antarblok perangkat lunak, menihilkan ancaman eksploitasi <i>buffer overflow</i> tanpa <i>overhead</i> signifikan.	Adopsi filosofi keamanan kompartemen menggunakan bahasa yang aman secara memori (Rust) wajib diterapkan untuk menjamin penyangga data (buffer) sensor kebal terhadap iterasi di luar batas.
10	Sensor Faults Isolation in Networked Control Systems: Application to Mobile Robot Platoons <i>Kurniawan, 2021</i>	Menyusun kerangka kerja diagnosis kerusakan menggunakan metode <i>Unknown Input Observers</i> (UIO) terdistribusi guna mendeteksi dan melokalisasi malfungsi komponen.	Metodologi ini memvalidasi kemampuannya mengisolasi cacat pembacaan pada satu sensor sehingga perambatan galat tidak merusak presisi matriks kontrol keseluruhan.	Integrasi algoritma isolasi kesalahan (<i>fault isolation</i>) waktu nyata diperlukan agar kerusakan pada satu bus akuisisi data tidak menghentikan keseluruhan proses di mikrokontroler.
11	An Analyzable Inter-core Communication Framework for High-Performance Multicore Embedded Systems <i>Tabish et al., 2021</i>	Memperkenalkan <i>Communication Core Model</i> (CCM) untuk memfasilitasi pertukaran pesan asinkron antar-inti dengan mengendalikan interferensi sumber daya bersama.	Kerangka kerja ini memuluskan transfer data pada topologi multiprosesor sembari memastikan kepastian waktu eksekusi tanpa risiko tabrakan akses (<i>data contention</i>).	Penerapan struktur ini pada prosesor <i>Dual-Core</i> ESP32-S3 akan memungkinkan pemisahan tugas murni: Core 0 untuk orkestrasi jaringan, dan Core 1 untuk akuisisi sensor kritikal.

Berlanjut ke halaman berikutnya...

Table 1 – *Lanjutan dari halaman sebelumnya*

No	Judul Artikel & Penulis (Tahun)	Metode	Hasil Utama	Future Work
12	A High-Performance, Energy-Efficient Modular DMA Engine Architecture <i>Benz et al., 2024</i>	Mendesain ulang blok <i>Direct Memory Access Engine</i> (DMAE) secara modular dengan mendekomposisi unit kontrol, akselerasi transfer matriks kompleks, dan antarmuka struktur (<i>fabric</i>).	Arsitektur iDMA ini mendemonstrasikan efisiensi termal dan operasional yang prima dalam merencanakan aliran data masif tanpa mengeksekusi instruksi dari prosesor utama.	Eksplorasi arsitektur DMA internal ESP32-S3 perlu dioptimalkan guna mengalirkan vektor data sensor ke blok SRAM secara absolut tanpa membebani interupsi CPU.
13	Arm cortex M4 microprocessors based $\pm$ 100 kVAR energy quality regulator for reactive power/neutral current compensation, load balancing and harmonic mitigation <i>Eren & Vural, 2022</i>	Memaksimalkan modul komputasi <i>Digital Signal Processing</i> (DSP) pada mikrokontroler seri ARM Cortex-M4 untuk meredam gangguan harmonisa secara <i>on-the-fly</i> .	Infrastruktur instruksi mikrokontroler memverifikasi ketangguhannya dalam mengeksekusi komputasi aljabar (<i>floating-point</i>) kompleks demi menjamin stabilitas data reaktif.	Sistem perlu mendayagunakan kapabilitas arsitektur komputasi vektor/DSP pada ESP32-S3 guna mengakselerasi komputasi algoritma filter derau digital pada sinyal mentah.
14	Providing a CoAP-based technique to get wireless sensor data via IoT gateway <i>Nikseresht & Mollamotalebi, 2021</i>	Mengajukan skema protokol transfer CoAP (<i>Constrained Application Protocol</i>) di atas spektrum 6LoWPAN untuk mengecilkan <i>overhead</i> muatan data (<i>payload</i>).	Optimasi ukuran bingkai paket data sukses mendongkrak efisiensi pita lebar (<i>bandwidth</i>) sekaligus memangkas konsumsi daya transmisi secara substansial pada <i>node</i> jaringan.	Aplikasi skema transmisi ringan (CoAP/UDP) direkomendasikan pada proses pengiriman paket data (<i>batching</i>) ESP32-S3 demi meminimalkan durasi aktif perifer radio (WiFi/BLE).

Berlanjut ke halaman berikutnya...

Table 1 – Lanjutan dari halaman sebelumnya

No	Judul Artikel & Penulis (Tahun)	Metode	Hasil Utama	Future Work
15	A zero-copy lock-free data pipeline for edge streaming <i>Su & Zhang, 2026</i>	Membangun infrastruktur <i>data pipeline</i> LZDP berbasis operasi atomik (<i>zero-copy</i> dan <i>lock-free</i>) untuk memenuhi tuntutan sistem analitik di komputasi sisi tepi (<i>edge</i>).	Konstruksi arsitektur ini membebaskan sistem dari duplikasi blok memori yang mubazir, mengurangi penundaan propagasi (<i>latency</i>), serta melipatgandakan <i>throughput</i> data.	Mekanisme pertukaran data <i>zero-copy</i> dan antrean bebas-kunci (<i>lock-free queue</i>) menggunakan Rust ideal diimplementasikan untuk menjembatani aliran data dari sensor langsung ke modul transmisi.

Berangkat dari sintesis 15 literatur di atas, beragam *future work* tersebut diposisikan sebagai referensi strategis dalam penentuan arah pengembangan sistem. Walaupun demikian, demi menjaga integritas antara dokumen laporan dengan eksekusi kode riil, tidak semua *future work* diadopsi pada fase ini. Implementasi saat ini difokuskan secara mendalam pada fusi kapabilitas bahasa Embedded Rust dengan periferan bawaan ESP32-S3, khususnya mencakup: pembacaan sensor digital dan analog, penyimpanan data pada *static circular buffer*, reduksi fluktuasi dengan filter *moving average*, rutinitas validasi data, logika *decision-making* berdasarkan ambang batas (threshold), aktuasi LED indikator, serta ekspos serial monitor berformat CSV.

Studi implementasi ini bertajuk **“Implementasi Embedded Rust pada ESP32-S3 untuk Akuisisi Data Sensor DHT22 dan MQ-2 dengan Circular Buffer, Moving Average, Validasi Data, Decision Logic, dan Kontrol LED.”** Arsitektur sistem mempekerjakan sensor DHT22 untuk mengekstrak parameter suhu (*TEMP*) dan kelembapan (*HUM*), serta sensor gas MQ-2 untuk mendeteksi indeks kualitas udara (*GAS*). Akuisisi sinyal digital sensor DHT22 dialokasikan pada GPIO 5, sedangkan sinyal analog dari sensor MQ-2 ditangkap oleh kanal ADC1 pada GPIO 4. Data mentah yang disampling selanjutnya dikonversi ke dalam satuan fisis aktual, disimpan ke dalam penyangga melingkar berkapasitas tetap, dirata-ratakan, dan dievaluasi guna menyimpulkan kondisi sistem saat ini.

Korelasi antara temuan studi literatur dan ruang lingkup implementasi dirangkum sebagai berikut:

1. **Keamanan Akses Periferan via Embedded Rust.** Prinsip keamanan memori Rust dieksekusi secara langsung melalui pemanfaatan *Hardware Abstraction Layer* (HAL). Modul `esp-idf-hal` dipergunakan untuk memfasilitasi interaksi periferan (ADC dan GPIO) melalui aturan kepemilikan data (ownership) yang aman.
2. **Akuisisi Sensor Periodik Sekuensial.** Mikrokontroler membaca parameter dari sensor digital dan analog secara bergantian pada setiap siklus. Periode *sampling* distabilkan pada interval 1000 ms memanfaatkan fungsi `FreeRtos::delay_ms` di dalam *main loop* utama.

3. **Manajemen Memori dengan *Circular Buffer*.** Rekam jejak lima iterasi terakhir ditampilkan pada *static array* [SensorData; 5] menggunakan indeks modular. Ketika kapasitas maksimal tercapai, penunjuk indeks diatur kembali ke posisi awal sehingga data usang akan tertimpa otomatis tanpa menyebabkan fragmentasi memori (heap free).
4. **Peredaman Derau dengan *Moving Average*.** Kumpulan data historis di dalam *circular buffer* dihitung nilai rata-rata aritmetiknya. Proses ini berhasil memitigasi fluktuasi sesaat (noise) sinyal sebelum dialirkan ke blok evaluasi logika keputusan.
5. **Validasi Data dan Deteksi Kerusakan (*Fault Detection*).** Rutinitas program memverifikasi kelayakan nilai fisis pembacaan (TEMP rentang -20 hingga 80°C , HUM rentang $0 - 100\%$, dan GAS rentang $0 - 1000$ PPM). Apabila terjadi kegagalan baca sinyal (driver failure) atau data keluar dari ambang batas valid fisis, sistem bergeser secara instan ke status Fault, menyalakan LED Merah, dan mengisolasi data rusak dari buffer.
6. **Decision Logic dan Aktuasi LED.** Vektor data terfilter dibandingkan secara berurutan terhadap batas toleransi kritis. Rentang aman memicu LED Hijau, ambang waspada memicu LED Kuning, sedangkan limit kritis (Bahaya) maupun kondisi rusak (Fault) memaksa LED Merah menyala. Seluruh hasil determinasi ini turut dieksport ke dalam format CSV serial untuk visualisasi data dengan GNUPlot.

Harap dicatat bahwa orkestrasi asynchronous tasks murni tingkat inti, pemanfaatan interkoneksi memori DMA, *hardware watchdog timer*, serta distribusi protokol komunikasi IoT nirkabel belum diaplikasikan pada iterasi sistem kali ini. Aspek-aspek tersebut diklasifikasikan sebagai peta jalan pengembangan selanjutnya begitu stabilitas fondasi Embedded Rust di atas arsitektur ESP32-S3 telah diverifikasi secara meyakinkan.

2 Metode Implementasi yang Diusulkan

2.1 Diagram Blok

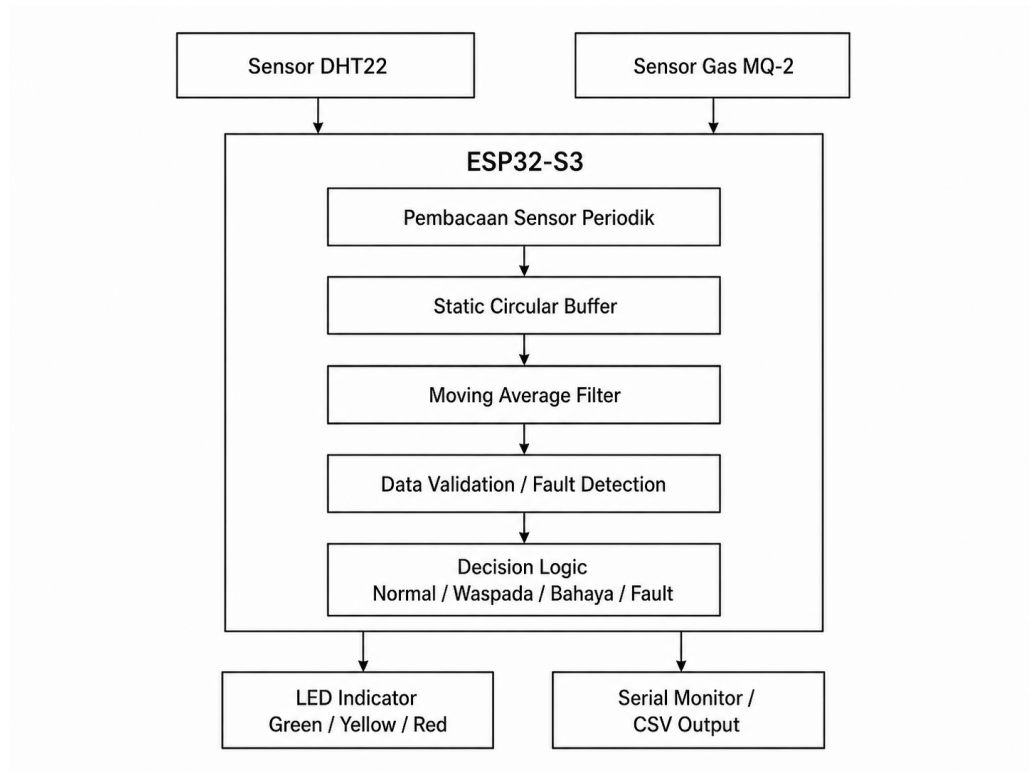


Figure 1: Diagram Blok Sistem Akuisisi Data Sensor DHT22 dan MQ-2

2.2 Flowchart

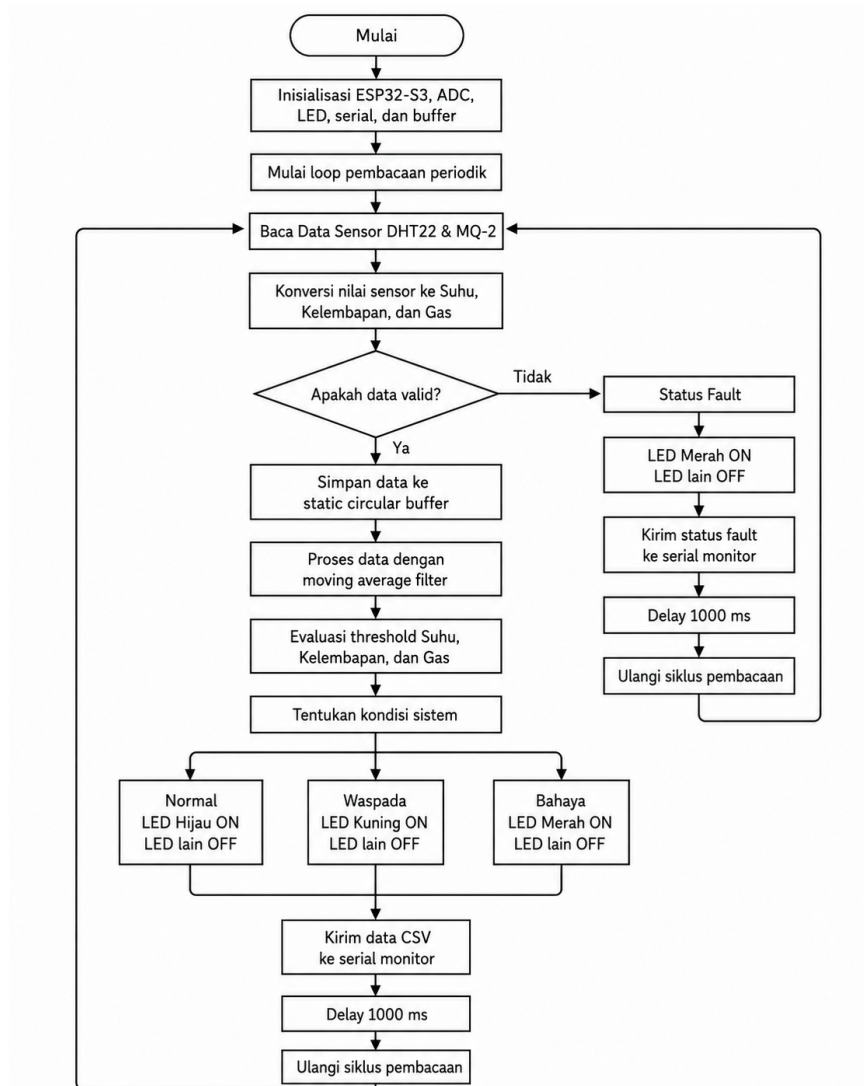


Figure 2: Flowchart Proses Akuisisi, Filtrasi, dan Decision Logic

2.3 Deskripsi Metode

Pendekatan ini menjabarkan implementasi *firmware* Rust tingkat tertanam pada *System-on-Chip* ESP32-S3 guna memfasilitasi pembacaan data sensor secara periodik. Parameter iklim mikro (suhu dan kelembapan) diakuisisi dari sensor digital DHT22 yang terhubung ke pin GPIO 5, sedangkan konsentrasi gas di udara dipantau menggunakan sensor analog MQ-2 yang dihubungkan ke pin analog GPIO 4 (ADC1 Kanal 3). Sebagai indikator visual luaran keputusan (decision logic), LED Hijau, Kuning, dan Merah masing-masing diintegrasikan pada pin GPIO 35, GPIO 36, dan GPIO 37.

Pada fase inisialisasi awal, struktur program mengonfigurasi periferal dasar sistem, mengaktifkan instansi ADC dengan atenuasi 11dB (rentang tegangan efektif 0–3.3 V), mendeklarasikan arah pin I/O, serta mengalokasikan ruang memori statis untuk komponen *circular buffer*. Program kemudian memberikan jeda waktu selama 2000 ms untuk menunggu kondisi elemen sensor DHT22 stabil sebelum memasuki perulangan (loop) utama.

Pada setiap iterasi, program mengekstrak data digital dari sensor DHT22 melalui fungsi

abstraksi driver bawaan. Secara simultan, nilai integer diskrit diakuisisi dari kanal ADC1 MQ-2. Nilai mentah ADC (skala 0 – 4095) ini direkayasa melalui formulasi konversi fisik pembagi tegangan untuk memperoleh nilai resistansi sensor (R_s) dan menetapkan tingkat kepekatan gas dalam satuan PPM.

Sebelum data dimasukkan ke dalam penampung, subsistem validasi memverifikasi status kelayakan rentang fisis data sensor. Apabila driver DHT22 mengembalikan nilai galat (error reading) atau data melompat melewati limit fisis mutlak, sistem langsung menetapkan status Fault. Kondisi kegagalan ini memicu pengisolasian data, pengaktifan LED Merah secara instan, serta pemaparan pesan peringatan pada Serial Monitor sebelum siklus diulang kembali.

Jika pengujian validasi sukses, paket data disimpan ke dalam static circular buffer berkapasitas 5 sampel data terakhir. Penambahan data diatur oleh perputaran indeks modular. Ketika alokasi array penuh, data pembacaan paling awal (oldest data) akan ditindih secara otomatis oleh data terbaru (newest data), menciptakan sistem manajemen memori FIFO melingkar tanpa alokasi dinamis.

Data yang terakumulasi di dalam buffer selanjutnya diperhalus melalui komputasi filter Simple Moving Average (SMA) untuk meminimalkan fluktuasi derau sinyal pembacaan. Nilai konklusif hasil filter inilah yang kemudian ditimbang terhadap aturan ambang batas keputusan (threshold logic). Status Waspada aktif jika suhu mencapai $\geq 25.0^\circ\text{C}$, kelembapan $\geq 30.0\%$, atau konsentrasi gas ≥ 300 PPM. Kondisi bereskalasi menjadi Bahaya apabila suhu melesat $\geq 35.0^\circ\text{C}$, kelembapan $\geq 45.0\%$, atau gas ≥ 600 PPM. Jika tren data berada di bawah batas waspada terkecil, operasional diklasifikasikan sebagai Normal.

Hasil final evaluasi status menggerakkan pin LED indikator yang bersesuaian dan mentransmisikan deret log berformat Comma-Separated Values (CSV) ke Serial Monitor. String CSV ini mengemas indeks nomor siklus, suhu, kelembapan, dan konsentrasi gas terfilter secara berurutan. Setiap siklus diakhiri dengan pemanggilan modul pewaktu `FreeRtos::delay_ms(1000)` sebelum sistem mengulangi rantai pembacaan periferal.

2.4 Rumus Pemrosesan Data

Subbagian berikut merinci penjabaran operasional matematika dalam memanipulasi pemrosesan sinyal digital, pembagi tegangan elektrik, pengolahan memori cincin, dan penentuan klasifikasi keputusan.

2.4.1 Konversi Karakteristik Fisik Sensor Gas (MQ-2)

Sensor MQ-2 bekerja memanfaatkan prinsip perubahan konduktivitas material akibat adsorpsi gas pada elemen sensor. Secara elektrik, modul sensor membentuk konfigurasi pembagi tegangan (voltage divider) dengan resistor beban (R_L) bernilai $10\text{ k}\Omega$. Tegangan output analog (V_{out}) ditangkap oleh ADC 12-bit ESP32-S3 (0 – 4095) dan dikonversi ke nilai tegangan riil melalui hubungan:

$$V_{out} = \frac{ADC_{raw}}{4095} \times V_{cc} \quad (1)$$

Berdasarkan rumus pembagi tegangan, hubungan antara tegangan keluaran (V_{out}) dengan resistansi internal sensor (R_s) diturunkan secara aljabar menjadi:

$$R_s = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right) \quad (2)$$

Konsentrasi gas aktual dalam satuan PPM dihitung dengan membandingkan nilai R_s terhadap nilai resistansi sensor di udara bersih ($R_0 = 9.83 \text{ k}\Omega$) memanfaatkan persamaan regresi non-linear eksponensial (Power-Law) berdasarkan kurva karakteristik datasheet pabrikan:

$$PPM = a \cdot \left(\frac{R_s}{R_0} \right)^b \quad (3)$$

Di mana nilai konstanta regresi yang digunakan adalah $a = 116.6020682$ dan eksponen kemiringan grafik $b = -2.769034857$.

2.4.2 Circular Buffer dan Moving Average

Penyimpanan tumpukan data pada memori cincin menggunakan larik berdimensi tetap dengan kedalaman maksimal 5 sampel. Penyesuaian variabel penunjuk indeks data baru dihitung berdasarkan operasi modulo:

$$i_{\text{baru}} = (i_{\text{lama}} + 1) \bmod 5 \quad (4)$$

Jumlah populasi elemen data (N) di dalam buffer yang divalidasi meningkat hingga batas jenuh maksimal 5:

$$N_{\text{baru}} = \min(N_{\text{lama}} + 1, 5) \quad (5)$$

Proses pemulusan data dari pengaruh derau transien dihitung menggunakan teorema filter digital Simple Moving Average (SMA) berikut:

$$\overline{TEMP} = \frac{1}{N} \sum_{i=1}^N TEMP_i \quad (6)$$

$$\overline{HUM} = \frac{1}{N} \sum_{i=1}^N HUM_i \quad (7)$$

$$\overline{GAS} = \frac{1}{N} \sum_{i=1}^N GAS_i \quad (8)$$

2.4.3 Validasi Kelayakan Data

Sebelum dievaluasi oleh pohon keputusan, data pembacaan diperiksa kevalidannya demi menjamin pemisahan kegagalan (fault isolation). Kriteria pemenuhan data sah didefinisikan sebagai:

$$\text{Valid} = (-20.0 \leq TEMP \leq 80.0) \wedge (0.0 \leq HUM \leq 100.0) \wedge (0.0 \leq GAS \leq 1000.0) \quad (9)$$

Jika data melanggar ketentuan batas fisis pada Persamaan (9), sistem secara otomatis mengabaikan paket pembacaan tersebut dan menetapkan status Fault.

2.4.4 Logika Penentuan Status Sistem

Penetapan status kondisi lingkungan dijalankan berdasarkan evaluasi bertingkat (conditional branching) terhadap hasil data terfilter:

$$\text{Status} = \begin{cases} \text{Fault,} & \text{Sensor terputus} \vee \neg \text{Valid,} \\ \text{Bahaya,} & \overline{TEMP} \geq 35.0 \vee \overline{HUM} \geq 45.0 \vee \overline{GAS} \geq 600.0, \\ \text{Waspada,} & \overline{TEMP} \geq 25.0 \vee \overline{HUM} \geq 30.0 \vee \overline{GAS} \geq 300.0, \\ \text{Normal,} & \text{selain kondisi di atas.} \end{cases} \quad (10)$$

2.4.5 Logika Aktuasi LED

Sistem merespons hasil keputusan status fisis dengan merubah kombinasi logika pin output digital menuju driver LED:

$$\text{LED} = \begin{cases} \text{Hijau ON, Kuning dan Merah OFF,} & \text{Status = Normal,} \\ \text{Kuning ON, Hijau dan Merah OFF,} & \text{Status = Waspada,} \\ \text{Merah ON, Hijau dan Kuning OFF,} & \text{Status = Bahaya atau Fault.} \end{cases} \quad (11)$$

3 Simulasi dan Hasil

3.1 Alat dan Perangkat Lunak

- Target Mikrokontroler Inti: **ESP32-S3**.
- Platform Eksekusi Digital Berbasis Web: **Wokwi Simulator**.
- Lapis Pemrograman Sistem: **Embedded Rust** (*crate esp-idf-hal*).
- Ruang Kerja Pengembangan (IDE): **Visual Studio Code**.
- Desain Skematik & Pratinjau Jaringan: **Proteus 9.1**.
- Mesin Ekstraksi Grafik Plot Data: **GNUPlot**.

3.2 Rangkaian Simulasi

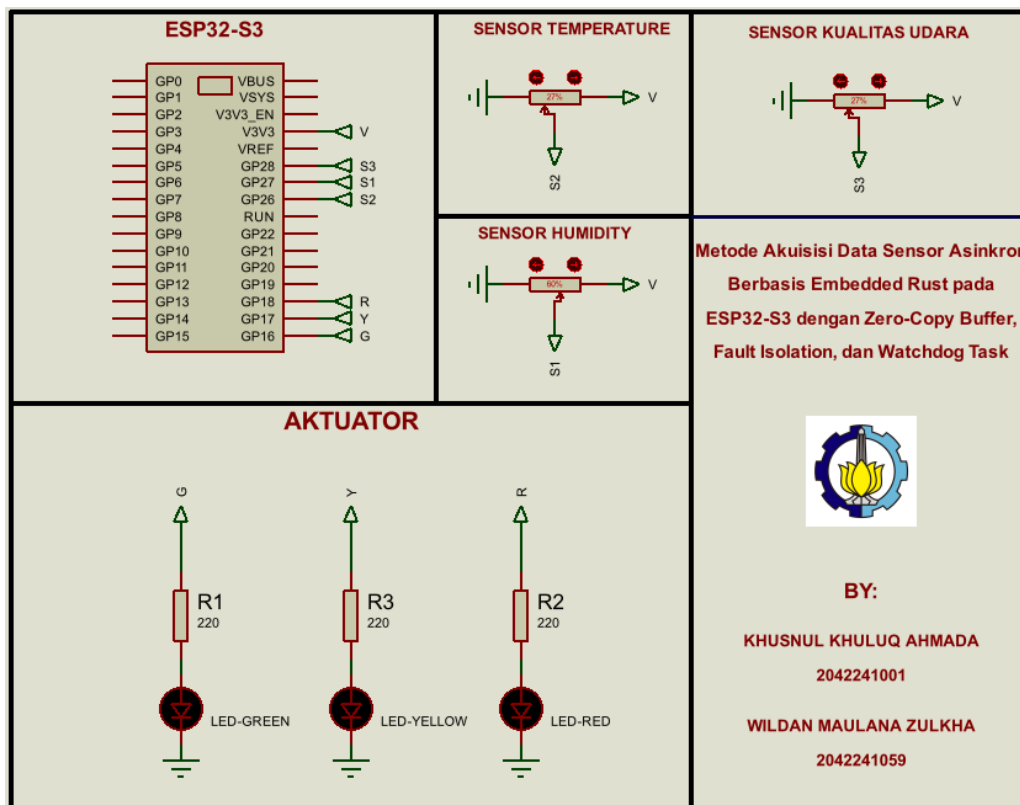


Figure 3: Visualisasi Desain Skematik Rangkaian Awal pada Proteus

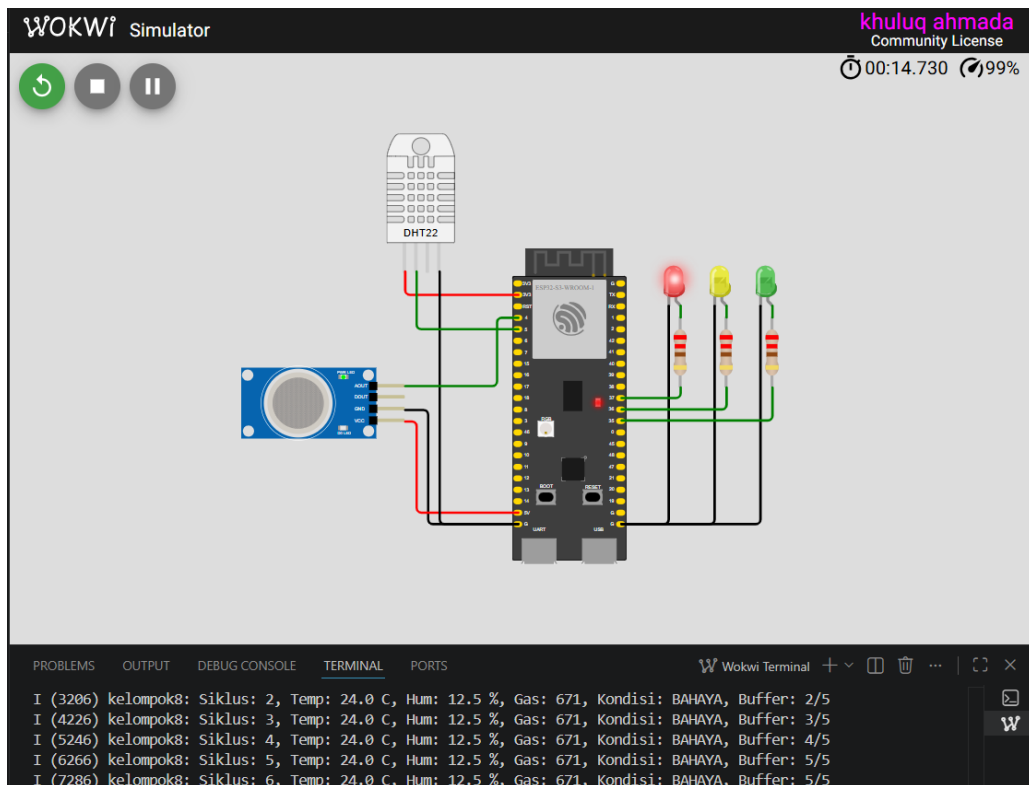


Figure 4: Simulasi Utama Menggunakan Sensor Fisik pada Wokwi

Konstruksi rangkaian elektronika dibangun mengintegrasikan cip ESP32-S3 DevKitC-1 bersama sensor digital DHT22 dan sensor gas analog MQ-2. Tiga lampu indikator LED dipasangkan seri dengan resistor beban 220 Ω sebagai langkah proteksi arus berlebih (current limiting). Jalur data sensor DHT22 dialokasikan secara eksklusif ke pin GPIO 5, sedangkan kaki keluaran analog MQ-2 disambungkan menuju pin analog GPIO 4. LED Hijau, Kuning, dan Merah masing-masing terhubung ke GPIO 35, 36, dan 37.

Fungsionalitas Proteus didelegasikan khusus sebagai instrumen pratinjau rancangan skematik awal. Lingkungan simulasi interaktif yang mengeksekusi biner (binary execution) kompilasi Rust secara riil dilimpahkan sepenuhnya kepada platform Wokwi. Pendekatan ini menjamin validasi pengujian pembacaan pulsa digital 1-Wire, akurasi sampling ADC, operasi logika, aktuasi pin LED, serta keandalan transmisi data log monitor serial berjalan tanpa deviasi.

3.3 Data dan Plot GNUPlot

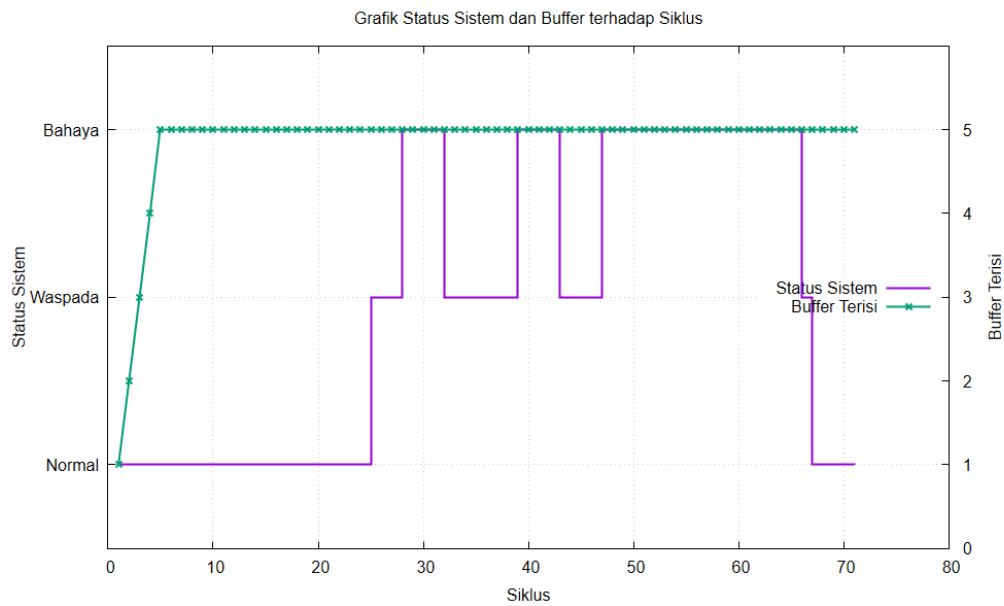


Figure 5: Grafik Status Keputusan Sistem dan Kedalaman Buffer terhadap Siklus

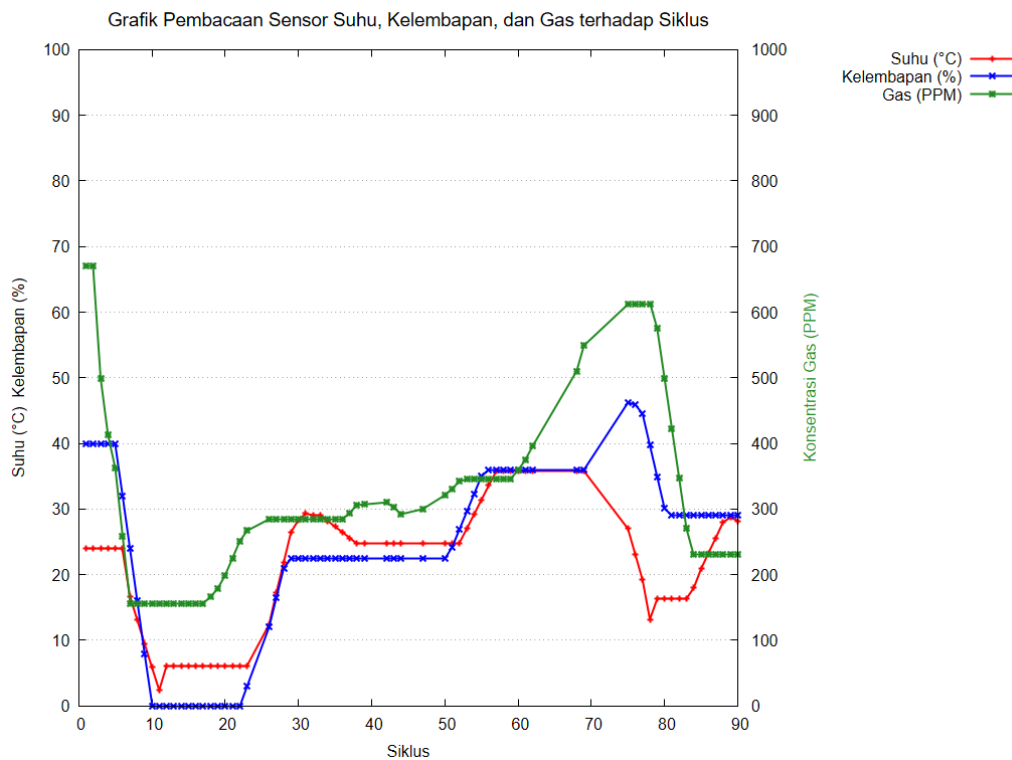


Figure 6: Grafik Hasil Pembacaan Parameter Sensor Hasil Plot GNUPlot

Gambar 6 menguraikan visualisasi data temporal dari hasil konversi fisis parameter TEMP, HUM, dan GAS. Parameter input diubah secara dinamis melalui fitur interaktif slider pada simulator Wokwi untuk memverifikasi ketangguhan respon transisi sistem terhadap fluktuasi

kondisi lingkungan. Seluruh titik data grafik direkonstruksi langsung dari penyerapan berkas *'data_sensor.txt'* berformat *CSV* yang dikirimkan oleh *ESP32 - S3*.

Gambar 5 memaparkan korelasi antara kedalaman akumulasi tumpukan data bufer terhadap status keputusan sistem. Pada awal siklus sistem menyala, grafik memaparkan kenaikan volume data hingga menyentuh batas kapasitas horizontal konstan ($N = 5$) akibat sifat circular buffer yang menindih histori terlama. Osilasi kurva keputusan merefleksikan perubahan dinamis dari kondisi Normal menuju Waspada dan Bahaya secara instan, mengonfirmasi keberhasilan performa algoritma decision logic monolitik yang dirancang.

4 Keunggulan Implementasi

4.1 Keamanan Memori Periferal via Embedded Rust

Nilai tambah utama dari proyek ini bermuara pada pemanfaatan arsitektur bahasa Rust untuk mendikte interaksi tingkat perangkat keras secara aman dan bebas dari celah kebocoran memori. Melalui pemanfaatan lapisan Hardware Abstraction Layer (HAL) dari pustaka `esp-idf-hal`, manipulasi register internal untuk konversi multi-channel ADC serta pemicuan pin output GPIO dapat dieksekusi secara terstruktur. Pola desain kode sumber pada ‘main.rs’ menerapkan enkapsulasi tipe data objek `SensorData` dan struktur modularitas fungsi, mewujudkan kode program yang bersih (clean code), mudah didebug, serta memiliki skalabilitas pengembangan yang tinggi.

4.2 Optimalisasi Arsitektur Static Circular Buffer

Penyimpanan jejak data historis diamankan melalui alokasi memori lokal berukuran tetap `[SensorData; 5]`. Penggunaan struktur data statis ini melenyapkan ancaman fragmen-tasi memori RAM serta resiko out-of-memory yang kerap ditemui pada inisiasi alokasi dinamis (heap memory) bahasa tradisional C/C++ dalam proses perulangan kontinu. Logika rotasi memori melingkar menjamin ketersediaan data aktual secara konstan untuk diolah oleh algoritma filter tanpa membebani kapasitas RAM mikrokontroler.

4.3 Filtrasi Derau, Validasi Data, dan Decision Logic

Sistem ini menerapkan filter digital Moving Average untuk menghaluskan riwayat data sebelum diserahkan pada logika pengambilan keputusan. Hal ini memitigasi anomali pembacaan akibat gangguan lonjakan sesaat (noise spike), sehingga kestabilan aktuasi indikator keluaran tetap terjaga. Ditambah dengan subsistem pre-processing validasi, data sensor yang bersifat tidak masuk akal atau rusak akan langsung diisolasi secara instan guna mencegah kegagalan berantai (fault isolation), memaksa mikrokontroler berpindah ke state Fault demi keamanan sistem.

4.4 Batasan Implementasi saat Ini

Seluruh operasi komputasi saat ini dieksekusi sebagai rutinitas sekuensial sinkron pada satu sumbu perulangan utama (single main loop thread). Implementasi sistem pada fase ini belum melibatkan kerangka kerja tugas asinkron (async executor), penjadwalan multi-tasking, penggunaan DMA (Direct Memory Access), pengaktifan komponen pengawas perangkat keras Watchdog Timer, serta chip pemancar radio WiFi/BLE. Hambatan batasan ini ditetapkan secara sengaja guna memverifikasi stabilitas fondasi interaksi Embedded Rust mendasar sebelum ditingkatkan ke fungsionalitas arsitektur yang lebih kompleks.

5 E. Dokumentasi GitHub dan LinkedIn

5.1 GitHub

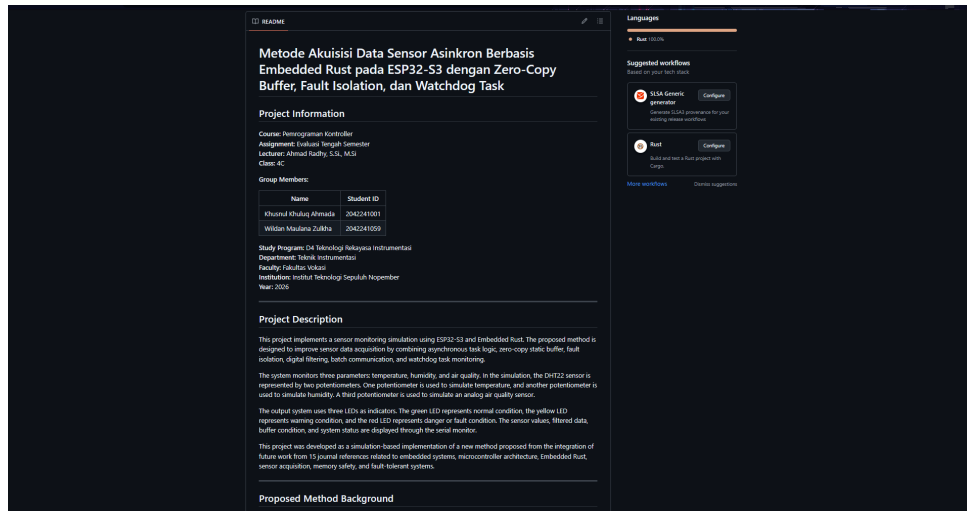


Figure 7: Repositori Publik GitHub Kelompok 8

Gudang penyimpanan berkas kode sumber (open-source repository) dapat diakses secara terbuka melalui tautan berikut:

<https://github.com/ahmada2135/kelompok8-esp32-s3-rust>

Repositori ini mengompilasi artefak kode Rust secara ekstensif, berkas konfigurasi komponen 'diagram.json', contoh data log CSV, skrip pemrosesan grafik GNUPlot, serta berkas instruksi teknis 'README.md'.

5.2 LinkedIn

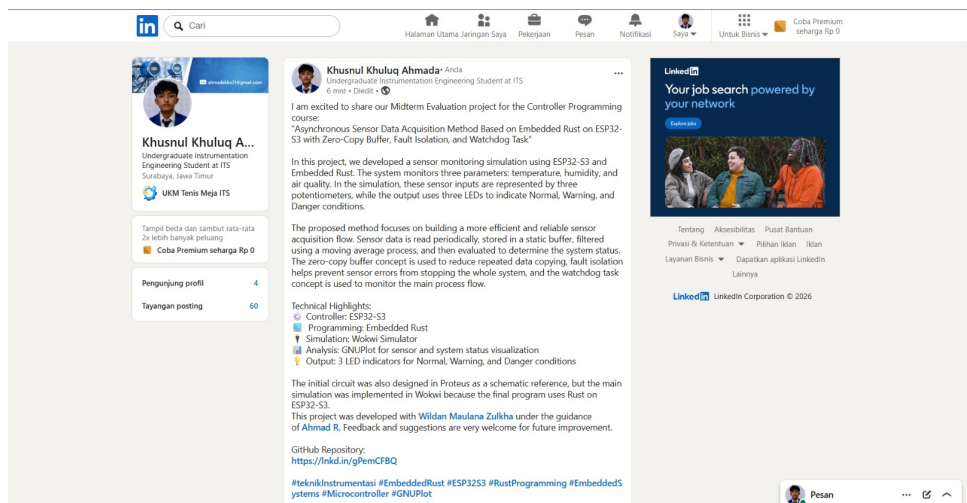


Figure 8: Publikasi dan Dokumentasi Akademik pada LinkedIn

Publikasi dokumentasi proyek melalui jaringan profesional memuat ringkasan metode kerja implementasi firmware Embedded Rust pada chip ESP32-S3, detail pemetaan pin sensor fisis DHT22 dan MQ-2, pengolahan filter perataan, serta visualisasi kurva pengujian performa sistem.

6 Kesimpulan

Merujuk pada paparan literatur, penguasaan metodologi *firmware*, observasi virtual via Wokwi, hingga derivasi grafik via GNUPlot, serangkaian konklusi solid dapat dirumuskan:

1. Bahasa pemrograman Embedded Rust terbukti berhasil diintegrasikan secara aman bersama periferan cip ESP32-S3 untuk mengontrol operasi multi-channel ADC analog serta akti-asi pin digital output GPIO.
2. Sistem akuisisi data telah sukses ditingkatkan dari rancangan awal berbasis potensiometer simulasi menjadi sistem pembacaan sensor fisik aktual secara riil menggunakan kompo-nen DHT22 dan MQ-2.
3. Penggunaan struktur Static Circular Buffer berkapasitas lima sampel mampu menghemat alokasi penggunaan memori RAM serta mencegah resiko terjadinya kebocoran memori pada mikrokontroler.
4. Filter digital Moving Average dan logika inspeksi validasi terbukti andal dalam mereduksi fluktuasi derau pembacaan sensor serta berhasil mengisolasi kegagalan data rusak (fault isolation).
5. Struktur Decision Logic mampu memisahkan tingkatan kondisi lingkungan secara kon-sisten ke dalam empat tingkatan status (Normal, Waspada, Bahaya, Fault) yang divisual-isasikan melalui kombinasi LED dan keluaran teks format CSV.
6. Proposisi sistem pada fase ini belum mengadopsi skema tugas asinkron, interkoneksi memori DMA, akselerasi perangkat keras watchdog timer, dan transmisi nirkabel WiFi/MQTT, yang diposisikan sebagai target peta jalan pengembangan proyek di masa mendatang.

Pustaka

- [1] S. Amar, T. Aviv, D. Chisnall, D. Chisnall, T. Chen, N. W. Filardo, R. Norton, S. W. Moore, S. Moore, Y. Tao, and R. N. M. Watson, “CHERIoT: Complete memory safety for embedded devices,” in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Toronto, ON, Canada, 2023, pp. 1–1, doi: 10.1145/3613424.3614266.
- [2] S. Automatisés, R. Mehalaine, M. Djezzar, D. Nessah, Z. Saiad, and A. Saidi, “Watchdog timer for fault tolerance in embedded systems,” *Journal Européen des Systèmes Automatisés*, vol. 57, no. 6, pp. 1713–1720, 2024.
- [3] T. Benz, M. Rogenmoser, P. Scheffler, S. Riedel, A. Ottaviano, A. Kurth, T. Hoefler, and L. Benini, “A high-performance, energy-efficient modular DMA engine architecture,” *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 263–277, 2024, doi: 10.1109/TC.2023.3329930.
- [4] I. Culic, A. Vochescu, and A. Radovici, “A low-latency optimization of a Rust-based secure operating system for embedded devices,” 2022.
- [5] A. Eren and A. M. Vural, “Arm Cortex-M4 microprocessors based ± 100 kVAR energy quality regulator for reactive power/neutral current compensation, load balancing and harmonic mitigation,” *Engineering Science and Technology, an International Journal*, vol. 27, p. 101018, 2022, doi: 10.1016/j.jestch.2021.05.022.
- [6] O. Gursoy, N. Akar, and S. Member, “Energy-age of incorrect information trade-off with timer-based sleep-wake scheduling,” 2024.
- [7] D. V. Hdez, P. Muñoz, D. F. Barrero, and M. D. R. Moreno, “Continuous energy consumption measure approach using a DMA double-buffering technique,” *EURASIP Journal on Wireless Communications and Networking*, 2021, doi: 10.1186/s13638-021-02043-w.
- [8] W. Kurniawan, “Sensor faults isolation in networked control systems,” pp. 1–21, 2021.
- [9] M. R. Nikseresht and M. Mollamotalebi, “Providing a CoAP-based technique to get wireless sensor data via IoT gateway,” *Computer Communications*, vol. 172, pp. 155–168, 2021, doi: 10.1016/j.comcom.2021.03.026.
- [10] A. Sharma, “Rust for embedded systems: Current state and open problems,” in *Proc. 2024 ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Salt Lake City, UT, USA, 2024, pp. 1–1, doi: 10.1145/3658644.3690275.
- [11] Y. Su and J. Zhang, “A zero-copy lock-free data pipeline for edge streaming,” *Information Sciences*, vol. 739, 2026, doi: 10.1016/j.ins.2026.123171.
- [12] R. Tabish, J. Wen, R. Pellizzoni, R. Mancuso, H. Yun, M. Caccamo, and L. Raymond, “An analyzable inter-core communication framework for high-performance multicore embedded systems,” *Journal of Systems Architecture*, vol. 118, p. 102178, 2021, doi: 10.1016/j.sysarc.2021.102178.
- [13] J. Trevathan, S. Schmidtke, W. Read, T. Sharp, and A. Sattar, “An IoT general-purpose sensor board for enabling remote aquatic environmental monitoring,” *Internet of Things*, vol. 16, p. 100429, 2021, doi: 10.1016/j.iot.2021.100429.

- [14] T. Vandervelden, R. De Smet, D. Deac, and K. Steenhaut, “Overview of embedded Rust operating systems and frameworks,” pp. 1–19, 2024.
- [15] N. Yoshimura, H. Oyama, and T. Azumi, “FMP3 + TECS/Rust: Memory-safe component framework for multiprocessor embedded systems,” *IEEE Open Journal of the Industrial Electronics Society*, vol. 7, pp. 469–487, 2026, doi: 10.1109/OJIES.2026.3666358.